Python Notes (Lecture 7)

File I/O

1. File I/O in Python

Python can be used to perform operations on a file: **reading** data from it and **writing** data to it. Data in variables is lost when a program ends; files store it permanently.

Types of files:

- **Text files**: store plain text, like `.txt`, `.log`, `.csv`, `.py`.
- **Binary files**: store data in binary form, like `.mp4`, `.mov`, `.png`, `.jpeg`.

Note: a Word file (`.docx`) is actually stored as a binary file, even though it contains text.

---

2. Open, Read and Close a File

We have to **open** a file before reading or writing, and **close** it when we are done.

```
f = open("file_name", "mode")
data = f.read()
f.close()
```

File modes:

Mode	Meaning
`'r'`	Open for reading (default)
`'w'`	Open for writing: erases the file first (creates it if missing)
`'x'`	Create a new file and open it for writing (error if it already exists)
`'a'`	Open for writing: adds to the end of the file (creates it if missing)
`'b'`	Binary mode
`'t'`	Text mode (default)
`'+'`	Open for updating (reading and writing)

Modes can be combined: `rb` reads a binary file, `r+` reads and writes.

---

3. Reading a File

Suppose `sample.txt` contains:

```
Hi everyone
we are learning File I/O
using Python.
```

Function	What it does
`f.read()`	Reads the entire file as one string
`f.readline()`	Reads one line at a time

```python
f = open("sample.txt", "r")
data = f.read()
print(data)
f.close()
```

Output:

```
Hi everyone
we are learning File I/O
using Python.
```

```python
f = open("sample.txt", "r")
line1 = f.readline()
line2 = f.readline()
print(line1, end="")
print(line2, end="")
f.close()
```

Output:

```
Hi everyone
we are learning File I/O
```

Each line read from a file ends with `\n` (new line), so we use `end=""` to avoid blank lines.

---

4. Writing to a File

`w` mode **overwrites** the entire file:

```
f = open("demo.txt", "w")
f.write("this is a new line")
f.close()
```

`a` mode **adds** to the end of the file:

```
f = open("demo.txt", "a")
f.write("\nthis line is added")
f.close()

f = open("demo.txt", "r")
print(f.read())
f.close()
```

Output:

```
this is a new line
this line is added
```

---

5. with Syntax

With `with`, Python **closes the file automatically** when the block ends, so we don't need `f.close()`.

```
with open("demo.txt", "r") as f:
    data = f.read()

print(data)
```

Output:

```
this is a new line
this line is added
```

```
with open("demo.txt", "w") as f:
    f.write("new data")
```

Note: to read a file, open it in `r` mode. Reading a file opened in `w` or `a` mode gives an error.

---

6. Deleting a File

We delete files using the **os module**. A **module** is a file (like a code library) written by another programmer that contains functions we can use.

```
import os

os.remove("demo.txt")
```

Let's Practice

Q1. Create a new file "practice.txt" using Python and add the following data in it:

Hi everyone
we are learning File I/O
using Java.
I like programming in Java.

```
with open("practice.txt", "w") as f:
    f.write("Hi everyone\nwe are learning File I/O\n")
    f.write("using Java.\nI like programming in Java.")
```

Q2. Write a function that replaces all occurrences of "java" with "python" in the above file.

```
def replace_java():
    with open("practice.txt", "r") as f:
        data = f.read()

    new_data = data.replace("Java", "Python")

    with open("practice.txt", "w") as f:
        f.write(new_data)

replace_java()

with open("practice.txt", "r") as f:
    print(f.read())
```

Output:

```
Hi everyone
we are learning File I/O
using Python.
I like programming in Python.
```

We first read the whole file, change the text, then write it back.

Q3. Search if the word "learning" exists in the file or not.

```
word = "learning"

with open("practice.txt", "r") as f:
    data = f.read()

if word in data:
    print("Found")
else:
    print("Not found")
```

Output:

```
Found
```

Q4. Write a function to find in which line of the file the word "learning" occurs first. Print -1 if not found.

```
def check_line(word):
    line_no = 1
    with open("practice.txt", "r") as f:
        for line in f:              # reads one line at a time
            if word in line:
                return line_no
            line_no += 1
    return -1

print(check_line("learning"))
print(check_line("hello"))
```

Output:

```
2
-1
```

Q5. From a file containing numbers separated by commas, print the count of even numbers.

Suppose `numbers.txt` contains: `1,2,3,4,5,6,7,8,9,10,12,15`

```
count = 0

with open("numbers.txt", "r") as f:
    data = f.read()

nums = data.split(",")       # list of strings
for num in nums:
    if int(num) % 2 == 0:
        count += 1

print("Even numbers:", count)
```

Output:

```
Even numbers: 6
```

`split(",")` breaks the string at every comma and returns a list.